

# Minería de Datos - Libro 1 - Clase 1 a 10

## 8. Aprendizaje Supervisado, Regresion

### Clase N° 8: Introducción al Aprendizaje Supervisado, Regresión Logística y Regresión Múltiple

#### Introducción

##### Objetivo de la Clase

El aprendizaje supervisado es una de las técnicas más utilizadas en minería de datos y aprendizaje automático. Se basa en entrenar un modelo utilizando **datos etiquetados**, es decir, datos en los que conocemos la respuesta correcta.

En esta clase, aprenderemos a:

- Entender qué es el aprendizaje supervisado y sus diferencias con el no supervisado.
- Distinguir entre problemas de clasificación y regresión.
- Explorar la regresión logística para problemas de clasificación binaria.
- Aplicar la regresión múltiple para predecir valores continuos.
- Evaluar modelos de regresión y clasificación en Python.

##### Aplicaciones del Aprendizaje Supervisado

- **Medicina:** Diagnóstico de enfermedades basado en síntomas (Clasificación).
- **Finanzas:** Predicción del valor de una acción en la bolsa (Regresión).
- **Marketing:** Identificación de clientes que responderán a una campaña publicitaria (Clasificación).
- **Industria Automotriz:** Predicción del consumo de combustible de un automóvil (Regresión).

**Ejemplo real:** Una aseguradora quiere predecir si un cliente presentará un reclamo el próximo año (Sí/No) → Regresión Logística. También puede querer predecir el monto del reclamo → Regresión Múltiple.

### Sección 1: Introducción al Aprendizaje Supervisado

#### Aprendizaje Supervisado vs. No Supervisado

| Tipo de Aprendizaje | Descripción                                                 | Ejemplo                                                   |
|---------------------|-------------------------------------------------------------|-----------------------------------------------------------|
| Supervisado         | Se entrena con datos donde se conoce la respuesta correcta. | Clasificación de correos en spam o no spam.               |
| No Supervisado      | El modelo encuentra patrones sin conocer etiquetas previas. | Agrupación de clientes según su comportamiento de compra. |

#### Diferencia entre Clasificación y Regresión

| Tipo de Problema | Descripción                         | Ejemplo                                          |
|------------------|-------------------------------------|--------------------------------------------------|
| Clasificación    | Predice una categoría o clase.      | Diagnóstico de enfermedades (Positivo/Negativo). |
| Regresión        | Predice un valor numérico continuo. | Predicción del precio de una casa.               |

#### Diagrama de Decisión: ¿Clasificación o Regresión?

- Si la variable a predecir es numérica continua → **Usar Regresión.**
- Si la variable a predecir es una categoría → **Usar Clasificación.**

#### 1.1. Implementación en Python – Clasificación vs. Regresión

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression, LinearRegression

# Datos sintéticos
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y_class = np.array([0, 0, 0, 1, 1, 1, 1, 1, 1, 1])
y_reg = np.array([2, 4, 5, 7, 10, 12, 14, 18, 20, 22])

# Ajustar modelos
clf = LogisticRegression().fit(X, y_class)
reg = LinearRegression().fit(X, y_reg)

X_test = np.linspace(0, 12, 100).reshape(-1, 1)
y_pred_class = clf.predict_proba(X_test)[:, 1]
y_pred_reg = reg.predict(X_test)

# Graficar
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X, y_class, color="blue", label="Datos")
plt.plot(X_test, y_pred_class, color="red", label="Modelo Logístico")
plt.title("Clasificación: Regresión Logística")
plt.legend()

plt.subplot(1, 2, 2)
plt.scatter(X, y_reg, color="blue", label="Datos")
plt.plot(X_test, y_pred_reg, color="red", label="Regresión Lineal")
plt.title("Regresión: Predicción de un Valor Continuo")
plt.legend()
plt.show()
```

### Conclusión:

- La Regresión Logística estima la probabilidad de una categoría (0 o 1).
- La Regresión Lineal estima un valor continuo.

### Ejercicio para la Sección 1

**Objetivo:** Diferenciar cuándo aplicar clasificación o regresión en un dataset real.

#### Instrucciones:

1. Carga el dataset del Titanic (`sns.load_dataset("titanic")`).
2. Define si la predicción de supervivencia es clasificación o regresión.
3. Aplica un modelo de Regresión Logística para predecir la supervivencia.
4. Aplica una Regresión Lineal para predecir la tarifa del boleto según la edad.
5. Explica qué modelo fue más apropiado en cada caso.

#### Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.model_selection import train_test_split

df = sns.load_dataset("titanic").dropna()

X_class = df[["age", "fare"]]
y_class = df["survived"]
X_reg = df[["age"]]
y_reg = df["fare"]

X_tr_c, X_te_c, y_tr_c, y_te_c = train_test_split(X_class, y_class, test_size=0.2, random_state=42)
X_tr_r, X_te_r, y_tr_r, y_te_r = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

clf = LogisticRegression(max_iter=1000).fit(X_tr_c, y_tr_c)
reg = LinearRegression().fit(X_tr_r, y_tr_r)

print(f"Precisión Logística: {clf.score(X_te_c, y_te_c):.2f}")
print(f"R² Lineal: {reg.score(X_te_r, y_te_r):.2f}")
```

Preguntas de Reflexión

- 1. ¿Por qué es más adecuado usar Regresión Logística para predecir la supervivencia?
- 2. ¿Por qué la Regresión Lineal es más apropiada para predecir la tarifa del boleto?
- 3. ¿Qué características adicionales podríamos incluir para mejorar los modelos?

Sección 2: Regresión Logística

2.2. Función Sigmoide

La función sigmoide convierte cualquier valor en un rango de 0 a 1, facilitando la interpretación como probabilidad.

- Si el resultado es cercano a **1**, el modelo predice clase 1 (ejemplo: "Aprobado").
- Si es cercano a **0**, predice clase 0 (ejemplo: "Rechazado").
- Se establece un umbral de clasificación, típicamente **0.5**.

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-10, 10, 100)
plt.plot(z, sigmoid(z), label="Sigmoide", color="blue")
plt.axvline(0, color="gray", linestyle="--", label="Umbral (0)")
plt.axhline(0.5, color="red", linestyle="--", label="Probabilidad = 0.5")
plt.xlabel("Z")
plt.ylabel("Probabilidad")
plt.title("Función Sigmoide en Regresión Logística")
plt.legend()
plt.show()
```

2.3. Evaluación del Modelo

Métricas para Regresión Logística

| Métrica               | Descripción                                                                              |
|-----------------------|------------------------------------------------------------------------------------------|
| Matriz de Confusión   | Muestra verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos. |
| Precisión             | Proporción de predicciones correctas sobre el total.                                     |
| Recall (Sensibilidad) | Capacidad del modelo para identificar correctamente la clase positiva.                   |
| Curva ROC-AUC         | Evalúa el desempeño comparando Tasa de Verdaderos Positivos vs. Falsos Positivos.        |

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report

y_real = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
y_pred = [1, 0, 1, 0, 0, 1, 0, 1, 1, 0]

cm = confusion_matrix(y_real, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Clase 0", "Clase 1"],
            yticklabels=["Clase 0", "Clase 1"])
plt.xlabel("Predicción")
plt.ylabel("Realidad")
plt.title("Matriz de Confusión")
plt.show()

print("
Reporte de Clasificación:
", classification_report(y_real, y_pred))
```

**Conclusión:** La matriz de confusión nos ayuda a ver los errores de clasificación. En salud, es mejor minimizar falsos negativos.

## Ejercicio para la Sección 2

**Objetivo:** Aplicar Regresión Logística en un problema real.

### Instrucciones:

1. Carga el dataset del Titanic.
2. Selecciona **age**, **fare** y **sex** como variables predictoras.
3. Transforma **sex** en variable numérica (0 = femenino, 1 = masculino).
4. Aplica Regresión Logística para predecir la supervivencia.
5. Evalúa el modelo con una matriz de confusión y precisión.

### Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = sns.load_dataset("titanic").dropna()
df["sex"] = df["sex"].map({"male": 1, "female": 0})
X = df[["age", "fare", "sex"]]
y = df["survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Precisión:", accuracy_score(y_test, y_pred))
print("
Matriz de Confusión:
", confusion_matrix(y_test, y_pred))
print("
Reporte:
", classification_report(y_test, y_pred))
```

### Preguntas de Reflexión

1. ¿Cómo afecta la variable "sexo" a la predicción de supervivencia?
2. ¿Por qué es importante balancear precisión y recall?
3. ¿Cómo podríamos mejorar el modelo agregando más variables?

## Sección 3: Regresión Múltiple

**Objetivo:** Comprender el modelo de Regresión Múltiple, cómo interpretar sus coeficientes y aplicarlo en un caso práctico.

3.1. ¿Qué es la Regresión Múltiple?

La Regresión Múltiple es una extensión de la Regresión Lineal donde se utilizan múltiples variables predictoras para estimar un valor numérico:

$$Y = \beta_0 + \beta_1X_1 + \beta_2X_2 + \dots + \beta_nX_n + \epsilon$$

- **Y**: variable dependiente (lo que queremos predecir).
- **X<sub>1</sub>, X<sub>2</sub>, ..., X<sub>n</sub>**: variables independientes.
- **β<sub>0</sub>**: intercepto.
- **β<sub>1</sub>, ..., β<sub>n</sub>**: coeficientes de cada variable predictora.
- **ε**: error del modelo.

3.2. Interpretación de los Coeficientes

| Variable                 | Coeficiente (β) | Interpretación                                                  |
|--------------------------|-----------------|-----------------------------------------------------------------|
| Superficie (m²)          | 500             | Por cada metro cuadrado adicional, el precio aumenta en \$500.  |
| Número de habitaciones   | 1500            | Por cada habitación adicional, el precio sube \$1500.           |
| Distancia al centro (km) | -2000           | Por cada kilómetro más lejos del centro, el precio baja \$2000. |

3.3. Multicolinealidad en Regresión Múltiple

Ocurre cuando dos o más variables independientes están fuertemente correlacionadas, afectando la estabilidad del modelo. Se detecta con la **matriz de correlación** o el **Factor de Inflación de Varianza (VIF)** — un VIF > 10 indica alta multicolinealidad.

```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor

data = {
    "Superficie":      [50, 60, 70, 80, 90, 100, 110, 120],
    "Habitaciones":    [2, 3, 3, 4, 4, 5, 5, 6],
    "Distancia_Centro":[5, 4, 3, 3, 2, 1, 1, 0]
}
df = pd.DataFrame(data)

vif_data = pd.DataFrame()
vif_data["Variable"] = df.columns
vif_data["VIF"] = [variance_inflation_factor(df.values, i) for i in range(df.shape[1])]
print(vif_data)
```

3.4. Evaluación del Modelo

Métricas para Regresión Múltiple

| Métrica                                  | Interpretación                                                                 |
|------------------------------------------|--------------------------------------------------------------------------------|
| <b>R² (Coeficiente de Determinación)</b> | Porcentaje de variabilidad explicada por el modelo (mayor es mejor, máximo 1). |
| <b>MSE (Error Cuadrático Medio)</b>      | Error promedio cuadrático (menor es mejor).                                    |
| <b>MAE (Error Absoluto Medio)</b>        | Promedio de los errores absolutos (menor es mejor).                            |

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

y_real = [200000, 220000, 250000, 270000, 300000]
y_pred = [210000, 225000, 240000, 260000, 295000]

print(f"R²: {r2_score(y_real, y_pred):.2f}")
print(f"MSE: {mean_squared_error(y_real, y_pred):.2f}")
print(f"MAE: {mean_absolute_error(y_real, y_pred):.2f}")
```

Ejercicio para la Sección 3

Instrucciones:

- 1. Carga un dataset de precios de viviendas.
- 2. Selecciona variables como superficie, habitaciones y baños como predictoras.
- 3. Aplica Regresión Múltiple para predecir el precio.
- 4. Evalúa el modelo con R<sup>2</sup>, MSE y MAE.
- 5. Analiza si hay multicolinealidad con VIF y corrige si es necesario.

Pista para la solución en Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Reemplazar con tu dataset real
# df = pd.read_csv("house_prices.csv")
# X = df[["sqft_living", "bedrooms", "bathrooms"]]
# y = df["price"]

# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# model = LinearRegression().fit(X_train, y_train)
# y_pred = model.predict(X_test)
# print(f"R²: {r2_score(y_test, y_pred):.2f}")
# print(f"MSE: {mean_squared_error(y_test, y_pred):.2f}")
# print(f"MAE: {mean_absolute_error(y_test, y_pred):.2f}")
```

Preguntas de Reflexión

- 1. ¿Qué variable tiene mayor impacto en el precio de la vivienda?
- 2. ¿Cómo afecta la multicolinealidad a la interpretación del modelo?
- 3. ¿Cómo podríamos mejorar el modelo?

Sección 4: Ejercicio Integrador y Publicación en el Foro Obligatorio

**Objetivo:** Aplicar los conceptos de Regresión Logística y Regresión Múltiple en un caso práctico, evaluar los resultados y compartir las conclusiones en el foro.

4.1. Ejercicio Integrador

**Contexto:** Trabajas en una empresa de seguros y necesitas desarrollar modelos para:

- Predecir si un cliente presentará un reclamo (Sí/No) → **Regresión Logística**.
- Estimar el monto del reclamo en dólares → **Regresión Múltiple**.

Dataset para el ejercicio

| Cliente | Edad | Ingresos (USD) | Historial de Accidentes | Monto Reclamo (USD) | Reclamo (Sí/No) |
|---------|------|----------------|-------------------------|---------------------|-----------------|
| 1       | 25   | 30000          | 1                       | 5000                | Sí              |
| 2       | 40   | 55000          | 0                       | 0                   | No              |
| 3       | 32   | 42000          | 1                       | 7500                | Sí              |
| 4       | 50   | 60000          | 0                       | 0                   | No              |
| 5       | 28   | 37000          | 1                       | 6500                | Sí              |
| 6       | 45   | 52000          | 0                       | 0                   | No              |

Tareas a realizar

- 1. **Preprocesamiento:** Convertir "Reclamo" a numérico (0=No, 1=Sí). Dividir en entrenamiento (80%) y prueba (20%).
- 2. **Regresión Logística:** Variables predictoras: Edad, Ingresos, Historial de Accidentes. Evaluar con Matriz de Confusión y Precisión.
- 3. **Regresión Múltiple:** Mismas variables predictoras. Evaluar con R<sup>2</sup>, MSE y MAE.
- 4. **Interacción con IA (LLM):** Consulta sugerida: "Tengo un dataset de aseguradoras. ¿Qué estrategias de modelado recomendarías para

*predecir el monto de un reclamo?"*

### Código Base para la Solución

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, r2_score,
                             mean_squared_error, mean_absolute_error)

data = {
    "Edad": [25, 40, 32, 50, 28, 45],
    "Ingresos": [30000, 55000, 42000, 60000, 37000, 52000],
    "Historial_Accidentes": [1, 0, 1, 0, 1, 0],
    "Monto_Reclamo": [5000, 0, 7500, 0, 6500, 0],
    "Reclamo": [1, 0, 1, 0, 1, 0]
}
df = pd.DataFrame(data)

X = df[["Edad", "Ingresos", "Historial_Accidentes"]]
y_c = df["Reclamo"]
y_r = df["Monto_Reclamo"]

X_tr_c, X_te_c, y_tr_c, y_te_c = train_test_split(X, y_c, test_size=0.2, random_state=42)
X_tr_r, X_te_r, y_tr_r, y_te_r = train_test_split(X, y_r, test_size=0.2, random_state=42)

# Regresión Logística
log_model = LogisticRegression(max_iter=1000).fit(X_tr_c, y_tr_c)
y_pred_c = log_model.predict(X_te_c)
print("Precisión:", accuracy_score(y_te_c, y_pred_c))
print("Matriz de Confusión:",
      confusion_matrix(y_te_c, y_pred_c))

# Regresión Múltiple
reg_model = LinearRegression().fit(X_tr_r, y_tr_r)
y_pred_r = reg_model.predict(X_te_r)
print(f"R²: {r2_score(y_te_r, y_pred_r):.2f}")
print(f"MSE: {mean_squared_error(y_te_r, y_pred_r):.2f}")
print(f"MAE: {mean_absolute_error(y_te_r, y_pred_r):.2f}")
```

### Bibliografía

1. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
2. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
3. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
4. McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.
5. Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.
6. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
7. Scikit-learn Developers. (n.d.). Recuperado de <https://scikit-learn.org/>